

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH I

**Nghiên cứu hệ thống tính toán song song
phân tán cho huấn luyện và suy luận mô hình
AI**

Nguyễn Hoàng

hoang.n250130e@sis.hust.edu.vn

20250130E

Ngành: Kỹ sư GenAI

Lớp: KS-GenAI-K69

GVHD: TS. Phạm Đăng Hải

Hà Nội, ngày tháng năm 2026

LỜI NÓI ĐẦU

Sự phát triển mạnh mẽ của Trí tuệ Nhân tạo (AI), đặc biệt là các Mô hình ngôn ngữ lớn, đã mang lại những đột phá sâu sắc trong nhiều lĩnh vực khoa học và đời sống. Tuy nhiên, đi kèm với đó là thách thức to lớn về mặt tính toán khi kích thước tham số của các mô hình này tăng theo cấp số nhân, vượt quá giới hạn vật lý và khả năng xử lý của các hệ thống máy tính đơn lẻ. Để giải quyết bài toán này, các hệ thống tính toán song song phân tán đã trở thành một nền tảng không thể thiếu.

Trong khuôn khổ môn học Đồ án chuyên ngành I, em đã chọn đề tài "**Nghiên cứu hệ thống tính toán song song phân tán cho huấn luyện và suy luận mô hình AI**". Báo cáo này là kết quả của quá trình thu thập, nghiên cứu và tổng hợp các kiến thức nền tảng từ cơ sở lý thuyết về lập trình song song trên CPU, GPU, đến các kỹ thuật tối ưu hóa giao tiếp liên thiết bị và các nền tảng phân tán tiên tiến nhất hiện nay.

Em xin gửi lời cảm ơn chân thành tới **TS. Phạm Đăng Hải** đã tận tình hướng dẫn, định hướng và đưa ra những lời khuyên quý báu giúp em tiếp cận và đi đúng hướng trong quá trình thực hiện đồ án. Dù đã cố gắng hết sức, nhưng do hạn chế về thời gian và kinh nghiệm thực tiễn, báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy cô để đồ án được hoàn thiện hơn và làm tiền đề cho các nghiên cứu chuyên sâu tiếp theo.

Hà Nội, ngày tháng năm 2026
Sinh viên thực hiện

Nguyễn Hoàng

LỜI CAM ĐOAN

Em xin cam đoan báo cáo đồ án chuyên ngành I với đề tài "Nghiên cứu hệ thống tính toán song song phân tán cho huấn luyện và suy luận mô hình AI" là kết quả của quá trình tự tìm hiểu, nghiên cứu và tổng hợp tài liệu dưới sự hướng dẫn của TS. Phạm Đăng Hải.

Các nội dung, kiến thức được trình bày trong báo cáo đều được tham khảo từ các nguồn tài liệu khoa học, bài báo và tài liệu kỹ thuật có nguồn gốc rõ ràng. Các trích dẫn đều được ghi rõ nguồn gốc trong phần Tài liệu tham khảo. Báo cáo không sao chép nguyên văn hay vi phạm quy định về đạo văn.

Nếu có bất kỳ sự gian lận nào, em xin chịu hoàn toàn trách nhiệm trước nhà trường và hội đồng đánh giá.

Hà Nội, năm 2026

Sinh viên thực hiện

Nguyễn Hoàng

MỤC LỤC

LỜI NÓI ĐẦU	2
DANH MỤC TỪ VIẾT TẮT	6
DANH MỤC HÌNH	7
DANH MỤC BẢNG	8
TÓM TẮT	9
1 Giới thiệu và cơ sở lý thuyết tính toán song song	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu và phạm vi nghiên cứu	1
1.3 Phương pháp nghiên cứu	1
1.4 Khái niệm cơ bản về lập trình song song	1
1.5 Cấu trúc báo cáo	3
2 Lập trình song song trên CPU: OpenMP và MPI	4
2.1 Giới thiệu chung	4
2.2 OpenMP: Mô hình bộ nhớ chia sẻ	4
2.3 MPI: Mô hình truyền thông điệp	4
2.4 Kết hợp OpenMP và MPI	5
2.5 So sánh OpenMP và MPI	5
3 Lập trình GPU và giao tiếp liên GPU	6
3.1 Tổng quan về GPU và Ứng dụng Deep Learning	6
3.2 Lập trình CUDA: Cấu trúc Host - Device	6
3.3 Kiến trúc phần cứng của GPU	6
3.4 Giao tiếp liên GPU: NCCL và NVSHMEM	8

4	Hệ thống huấn luyện và suy luận phân tán	10
4.1	Tổng quan về quá trình huấn luyện và suy luận mô hình AI	10
4.1.1	Các Framework huấn luyện phân tán	12
4.1.2	Các Engine suy luận phân tán	12
4.2	Các chiến lược song song hóa	13
4.3	Tối ưu hóa quá trình suy luận Inference Optimization	14
4.3.1	Tối ưu hóa trong pha Prefill	15
4.3.2	Tối ưu hóa quản lý bộ nhớ đệm KV Cache	16
4.3.3	Tối ưu hóa trong pha Decode	17
4.4	Kết luận	19
5	Thí nghiệm minh họa (Dự kiến)	20
	TÀI LIỆU THAM KHẢO	21
	PHỤ LỤC	23

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
CP	Context Parallelism	Song song theo ngữ cảnh
CPU	Central Processing Unit	Bộ xử lý trung tâm
CUDA	Compute Unified Device Architecture	Nền tảng tính toán song song của NVIDIA
DDP	Distributed Data Parallel	Song song dữ liệu phân tán
DP	Data Parallelism	Song song dữ liệu
EP	Expert Parallelism	Song song chuyên gia (trong MoE)
FSDP	Fully Sharded Data Parallel	Song song dữ liệu phân mảnh hoàn toàn
GPGPU	General-Purpose computing on GPU	Tính toán đa dụng trên GPU
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
HBM	High Bandwidth Memory	Bộ nhớ băng thông cao
MLA	Multi-head Latent Attention	Cơ chế tự chú ý ẩn đa đầu
MoE	Mixture of Experts	Hỗn hợp các chuyên gia
MPI	Message Passing Interface	Giao diện truyền thông điệp
MTP	Multi-Token Prediction	Dự đoán đa token
NCCL	NVIDIA Collective Communications Library	Thư viện giao tiếp tập thể của NVIDIA
NVSHMEM	NVIDIA Shared Memory	Thư viện bộ nhớ dùng chung của NVIDIA
OpenMP	Open Multi-Processing	Giao diện lập trình ứng dụng đa luồng
PGAS	Partitioned Global Address Space	Không gian địa chỉ toàn cục phân vùng
PP	Pipeline Parallelism	Song song đường ống
SIMD	Single Instruction, Multiple Data	Một lệnh, đa dữ liệu
SM	Streaming Multiprocessor	Bộ đa xử lý luồng
SRAM	Static Random-Access Memory	Bộ nhớ truy cập ngẫu nhiên tĩnh
TP	Tensor Parallelism	Song song theo Tensor
TTFT	Time-to-First-Token	Thời gian tạo token đầu tiên
VRAM	Video Random Access Memory	Bộ nhớ truy cập ngẫu nhiên video
ZeRO	Zero Redundancy Optimizer	Trình tối ưu hóa triệt tiêu dư thừa

DANH MỤC HÌNH

Figure 1.1	Cấu trúc máy tính von Neumann truyền thống	2
Figure 1.2	Phân loại kiến trúc máy tính theo Flynn	2
Figure 1.3	Mô hình bộ nhớ chia sẻ (Shared Memory)	3
Figure 1.4	Mô hình bộ nhớ chia sẻ không đồng nhất (NUMA - Hybrid) . . .	3
Figure 1.5	Mô hình bộ nhớ phân tán (Distributed Memory)	3
Figure 2.6	Cơ chế Fork-Join sinh luồng trong OpenMP	4
Figure 2.7	Mô hình giao tiếp qua truyền thông điệp MPI	5
Figure 3.8	Mô hình giao tiếp Host - Device trong kiến trúc CUDA	6
Figure 3.9	Kiến trúc phần cứng tiêu biểu của một vi xử lý GPU (CUDA) . .	7
Figure 3.10	Cấu trúc phân cấp Grid, Block và Thread trong CUDA	8
Figure 3.11	Giao tiếp tập thể sử dụng thư viện NCCL trên nhiều GPU	8
Figure 3.12	Mô hình bộ nhớ chia sẻ toàn cục phân vùng PGAS của NVSHMEM	9
Figure 4.13	Sự khác biệt về đặc thù hệ thống và tài nguyên giữa quá trình huấn luyện và suy luận	11
Figure 4.14	Kiến trúc quản lý và điều phối tài nguyên của Ray Train trong huấn luyện phân tán	12
Figure 4.15	Sơ đồ phân bổ và quản lý KV Cache của vLLM sử dụng cơ chế PagedAttention	13
Figure 4.16	Kiến trúc của SGLang	13
Figure 4.17	Các cấp độ tối ưu hóa bộ nhớ của DeepSpeed ZeRO	14
Figure 4.18	Cơ chế chia khối Tiling và tính toán trực tiếp trên SRAM của FlashAttention	15
Figure 4.19	Kiến trúc sinh token song song của Medusa sử dụng Tree Attention	18
Figure 4.20	Cơ chế Speculative Decoding ở cấp độ Feature của EAGLE . . .	19

DANH MỤC BẢNG

Table 2.1	So sánh đặc tính giữa OpenMP và MPI	5
Table 3.2	Quy trình 5 bước cơ bản trong lập trình CUDA	6
Table 4.3	So sánh đặc điểm hệ thống giữa Huấn luyện và Suy luận	11
Table 4.4	Tổng hợp các chiến lược song song hóa trên mô hình AI	14

TÓM TẮT

Đồ án chuyên ngành I tập trung nghiên cứu lý thuyết và kiến trúc của các hệ thống tính toán song song phân tán, đóng vai trò cốt lõi trong việc huấn luyện và suy luận các mô hình Trí tuệ Nhân tạo (AI) quy mô lớn hiện nay. Nội dung báo cáo được tổ chức thành bốn phần chính, đi từ các khái niệm cơ bản về lập trình song song trên CPU với OpenMP và MPI, đến việc khai thác sức mạnh của kiến trúc phần cứng GPU thông qua lập trình CUDA. Tiếp đó, báo cáo phân tích sâu các cơ chế giao tiếp liên GPU tiên tiến như NCCL và NVSHMEM, vốn là nền tảng cho việc luân chuyển dữ liệu ở tốc độ cao. Trọng tâm của đồ án nằm ở việc khảo sát các chiến lược song song hóa (Data Parallelism, Tensor Parallelism, Pipeline Parallelism, Context Parallelism) và cách chúng được hiện thực hóa trong các framework hàng đầu như PyTorch Distributed, DeepSpeed, và Megatron Core. Cuối cùng, sự khác biệt về yêu cầu phần cứng, bộ nhớ và băng thông mạng giữa hai giai đoạn huấn luyện (training) và suy luận (inference) cũng được làm rõ. Đồ án cung cấp một cái nhìn toàn cảnh, tạo cơ sở vững chắc cho việc triển khai và tối ưu hóa các hệ thống AI phân tán trong thực tế.

1 Giới thiệu và cơ sở lý thuyết tính toán song song

1.1 Đặt vấn đề

Sự bùng nổ của các mô hình Trí tuệ Nhân tạo (AI), đặc biệt là các mô hình ngôn ngữ lớn và Generative AI, đã vượt quá khả năng xử lý của các hệ thống máy tính đơn lẻ. Kích thước tham số của các mô hình này tăng theo cấp số nhân, đòi hỏi một lượng lớn bộ nhớ (VRAM) và năng lực tính toán. Điều này dẫn đến sự cần thiết tất yếu của các hệ thống tính toán song song phân tán nhằm phân tải khối lượng công việc, tối ưu hóa thời gian huấn luyện và đáp ứng tốc độ phản hồi trong quá trình suy luận.

1.2 Mục tiêu và phạm vi nghiên cứu

Đồ án được thực hiện với hai mục tiêu chính:

- Tổng hợp kiến thức nền tảng về tính toán song song và các framework phân tán tiên tiến hiện nay.
- Đề xuất và xây dựng một khung đánh giá hiệu năng (thí nghiệm) để kiểm chứng khả năng của các chiến lược song song hóa.

Phạm vi của báo cáo hiện tại tập trung ưu tiên nghiên cứu và tổng hợp cơ sở lý thuyết, phân tích cơ chế hoạt động của phần cứng và phần mềm. Phần thực nghiệm minh họa được cấu trúc dưới dạng khung dự kiến nhằm định hướng cho giai đoạn nghiên cứu và phát triển tiếp theo.

1.3 Phương pháp nghiên cứu

Phương pháp chủ đạo được áp dụng là nghiên cứu, thu thập và phân tích tài liệu kỹ thuật từ các nguồn uy tín như tài liệu của Phòng thí nghiệm Quốc gia Lawrence Livermore (LLNL), các ấn bản nghiên cứu trên arXiv, và mã nguồn mở từ NVIDIA, Microsoft, Meta. Dựa trên đó, đồ án thực hiện phân tích so sánh để làm rõ đặc tính của từng chiến lược song song hóa cũng như sự khác biệt giữa hai pha: huấn luyện (training) và suy luận (inference).

1.4 Khái niệm cơ bản về lập trình song song

Để hiểu rõ hệ thống phân tán, cần điểu qua các khái niệm cốt lõi:

- **Kiến trúc máy tính:** Từ cấu trúc von Neumann truyền thống phân tách CPU và bộ nhớ, đến phân loại Flynn (như SIMD - Single Instruction, Multiple Data và MIMD

- Multiple Instruction, Multiple Data) giải thích cách dữ liệu và luồng chỉ thị tương tác.

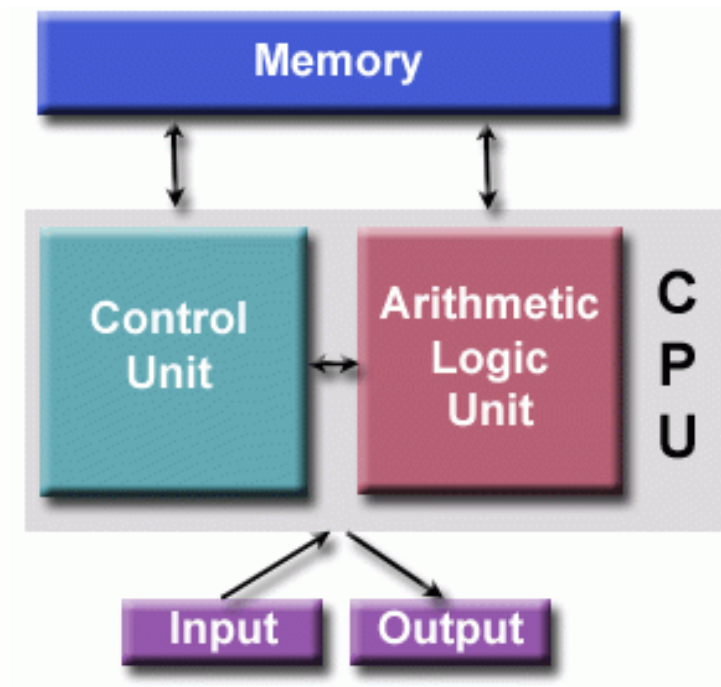


Figure 1.1 Cấu trúc máy tính von Neumann truyền thống

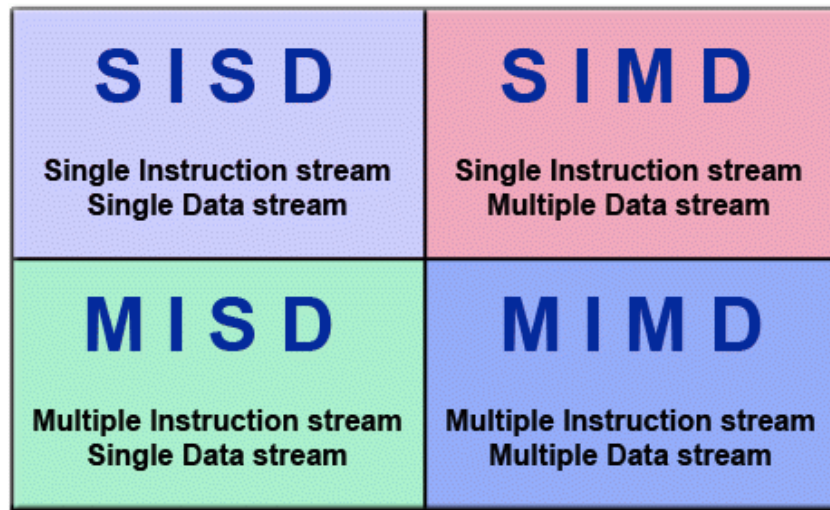


Figure 1.2 Phân loại kiến trúc máy tính theo Flynn

- **Kiến trúc bộ nhớ:** Gồm hệ thống bộ nhớ chia sẻ (Shared Memory), hệ thống bộ nhớ phân tán (Distributed Memory) và các mô hình lai (Hybrid) thường thấy trên các siêu máy tính hiện đại.

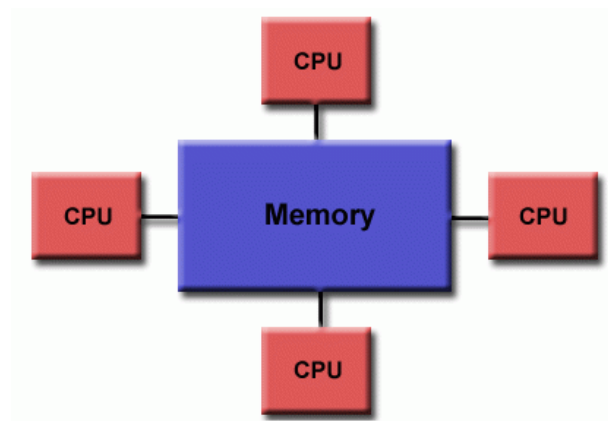


Figure 1.3 Mô hình bộ nhớ chia sẻ (Shared Memory)

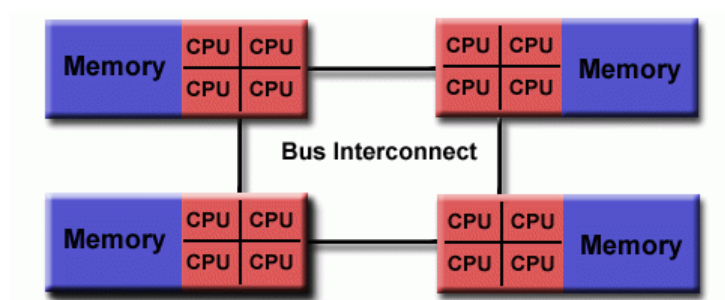


Figure 1.4 Mô hình bộ nhớ chia sẻ không đồng nhất (NUMA - Hybrid)

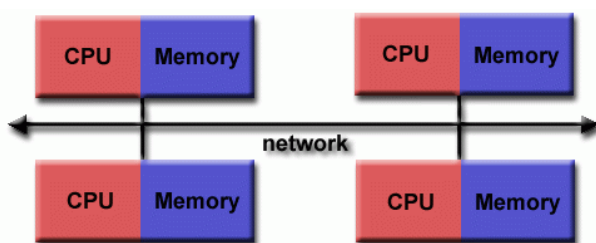


Figure 1.5 Mô hình bộ nhớ phân tán (Distributed Memory)

- **Mô hình lập trình:** Phổ biến nhất là OpenMP (hướng tới bộ nhớ chia sẻ) và MPI (hướng tới giao tiếp phân tán).

1.5 Cấu trúc báo cáo

Báo cáo được chia thành 4 phần chính. Phần 1 trình bày tổng quan và cơ sở lý thuyết. Phần 2 khảo sát các kỹ thuật lập trình song song trên CPU. Phần 3 đi sâu vào kiến trúc phần cứng GPU và các cơ chế giao tiếp liên GPU. Cuối cùng, Phần 4 phân tích các framework, các chiến lược phân tán cho AI, đưa ra so sánh giữa huấn luyện và suy luận, đồng thời đề xuất khung thực nghiệm minh họa.

2 Lập trình song song trên CPU: OpenMP và MPI

2.1 Giới thiệu chung

Trước khi các hệ thống gia tốc bằng phần cứng như GPU trở nên phổ biến, lập trình song song trên CPU đóng vai trò xương sống cho tính toán hiệu năng cao. Hai chuẩn thư viện phổ biến nhất đại diện cho hai mô hình bộ nhớ khác nhau là OpenMP và MPI.

2.2 OpenMP: Mô hình bộ nhớ chia sẻ

OpenMP (Open Multi-Processing) được thiết kế cho các hệ thống kiến trúc bộ nhớ chia sẻ (Shared Memory). Nó vận hành dựa trên cơ chế Fork-Join: chương trình bắt đầu với một luồng chính (Master Thread), sau đó phân nhánh (Fork) thành nhiều luồng con để xử lý song song các tác vụ, và gộp lại (Join) khi hoàn thành. Để tránh xung đột dữ liệu khi dùng chung không gian bộ nhớ, OpenMP cho phép lập trình viên định nghĩa các biến chia sẻ (shared) hoặc riêng tư (private). Các biến riêng tư sẽ được cấp phát các bản sao độc lập cho mỗi luồng, đảm bảo quá trình tính toán không bị gián đoạn.

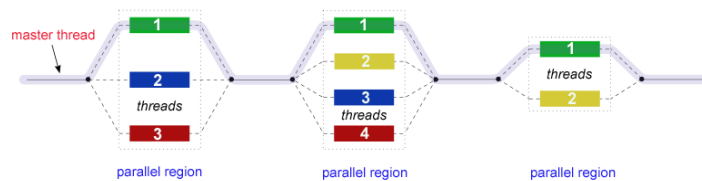


Figure 2.6 Cơ chế Fork-Join sinh luồng trong OpenMP

2.3 MPI: Mô hình truyền thông điệp

Khác với OpenMP, MPI (Message Passing Interface) là chuẩn lập trình cho các hệ thống bộ nhớ phân tán (Distributed Memory), nơi mỗi tiến trình sở hữu một vùng nhớ độc lập. MPI sử dụng cơ chế Communicators và Rank để định danh và quản lý các tiến trình. Việc giao tiếp giữa các tiến trình được thực hiện thông qua truyền thông điệp, bao gồm giao tiếp điểm - điểm (Point-to-Point) và giao tiếp tập thể (Collective Communication).

Message passing interface (MPI)

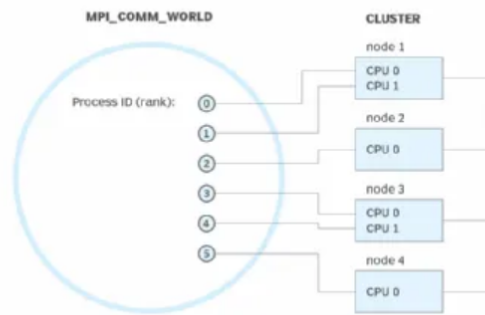


Figure 2.7 Mô hình giao tiếp qua truyền thông điệp MPI

2.4 Kết hợp OpenMP và MPI

Trên các siêu máy tính hiện đại với kiến trúc lai (Hybrid), việc kết hợp cả hai mô hình này là rất phổ biến: OpenMP được dùng để tối ưu hóa đa luồng trên các lõi CPU bên trong cùng một node (máy chủ), trong khi MPI chịu trách nhiệm kết nối và truyền tải dữ liệu giữa các node khác nhau.

2.5 So sánh OpenMP và MPI

Bảng 2.1 tổng hợp các điểm khác biệt cốt lõi giữa hai phương pháp lập trình này.

Table 2.1 So sánh đặc tính giữa OpenMP và MPI

Đặc điểm OpenMP	Đặc điểm MPI
Hoạt động trên mô hình bộ nhớ chia sẻ (Shared Memory).	Hoạt động trên mô hình bộ nhớ phân tán (Distributed Memory).
Cơ chế Fork-Join sinh luồng từ một Master Thread.	Cơ chế Communicator và Rank để quản lý các tiến trình độc lập.
Giao tiếp ngầm thông qua vùng nhớ chung (cần quản lý biến shared/private).	Giao tiếp tường minh thông qua truyền tin nhắn (Point-to-Point, Collective).
Được sử dụng chủ yếu trong nội bộ một node.	Phù hợp mở rộng quy mô tính toán giữa nhiều máy chủ (inter-node).

3 Lập trình GPU và giao tiếp liên GPU

3.1 Tổng quan về GPU và Ứng dụng Deep Learning

Mặc dù ban đầu được thiết kế cho đồ họa, kiến trúc nhiều lõi tính toán song song dựa trên mô hình SIMD (Single Instruction, Multiple Data) của GPU đã biến nó trở thành GPGPU (General-Purpose computing on GPU). Khả năng này cực kỳ phù hợp với các phép toán ma trận lớn, vốn là hạt nhân của quá trình huấn luyện Deep Learning.

3.2 Lập trình CUDA: Cấu trúc Host - Device

Trong hệ sinh thái CUDA do NVIDIA phát triển, phần cứng tính toán được chia làm hai khu vực độc lập: Host (bao gồm CPU và RAM hệ thống) và Device (GPU và VRAM). Hai thành phần này giao tiếp với nhau qua bus PCIe.

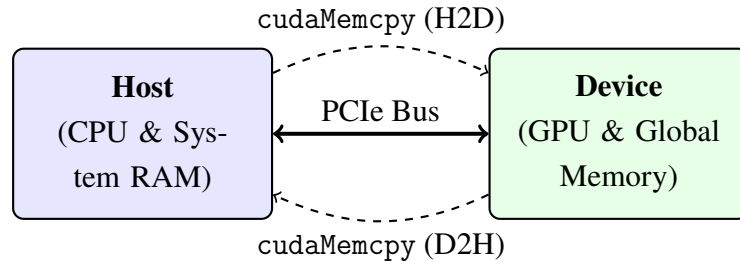


Figure 3.8 Mô hình giao tiếp Host - Device trong kiến trúc CUDA

Việc khởi chạy tính toán trên GPU tuân theo quy trình 5 bước cơ bản, được trình bày chi tiết trong Bảng 3.2.

Table 3.2 Quy trình 5 bước cơ bản trong lập trình CUDA

Bước	Lệnh CUDA	Mô tả thao tác
1	cudaMalloc	Cấp phát bộ nhớ cho Device (Global Memory).
2	cudaMemcpy	Sao chép dữ liệu đầu vào từ Host sang Device.
3	Kernel Launch	Khởi chạy tiến trình tính toán song song trên GPU.
4	cudaMemcpy	Lấy kết quả từ Device về lại Host.
5	cudaFree	Giải phóng bộ nhớ VRAM để tránh rò rỉ.

3.3 Kiến trúc phần cứng của GPU

Một vi xử lý GPU bao gồm nhiều Streaming Multiprocessors (SM). Mỗi SM chứa hàng loạt lõi xử lý (Cores/ALUs), bộ nhớ chia sẻ (Shared Memory/L1 Cache). Các luồng

thực thi được nhóm lại thành từng cụm 32 luồng gọi là Warp để thi hành chung một lệnh. Về mặt tổ chức logic do CUDA định nghĩa, quá trình tính toán phân cấp từ cao xuống thấp theo cấu trúc: **Grid** → **Block** → **Thread**. Toàn bộ các luồng đều dùng chung bộ nhớ L2 Cache trung tâm và bộ nhớ toàn cục (Global Memory/VRAM).

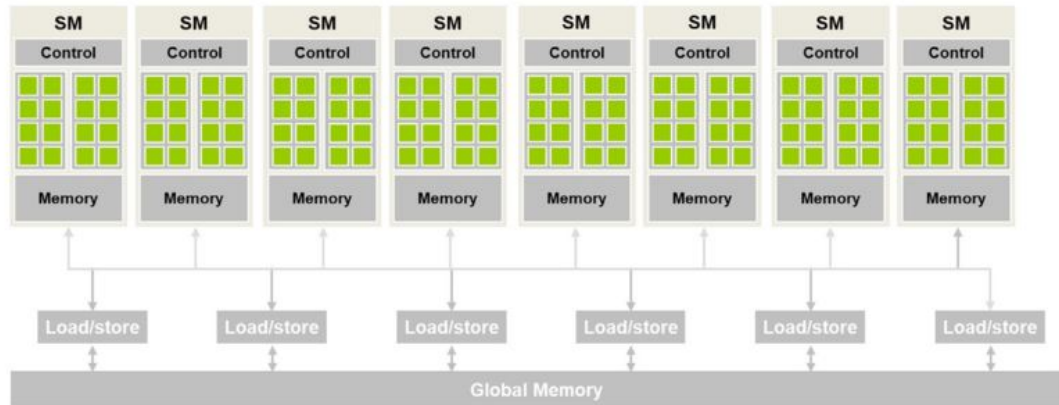


Figure 3.9 Kiến trúc phần cứng tiêu biểu của một vi xử lý GPU (CUDA)

Cấu trúc tính toán thành 3 tầng từ vĩ mô đến vi mô:

- **Grid:** Cấp độ quản lý cao nhất, chứa toàn bộ các Blocks được sinh ra trong một lần gọi hàm Kernel. Các Blocks trong cùng một Grid hoạt động hoàn toàn độc lập, không thể đồng bộ hóa chéo giữa Block này với Block khác.
- **Block:** Một nhóm chứa nhiều Threads, chúng có thể đồng bộ hóa lịch trình với nhau, và dùng chung một vùng nhớ tốc độ cực cao gọi là Shared Memory.
- **Thread:** Đơn vị thực thi nhỏ nhất và cơ bản nhất. Mỗi luồng sở hữu các thanh ghi (Registers) và vùng Local Memory hoàn toàn độc lập, không luồng nào khác có thể đọc được.

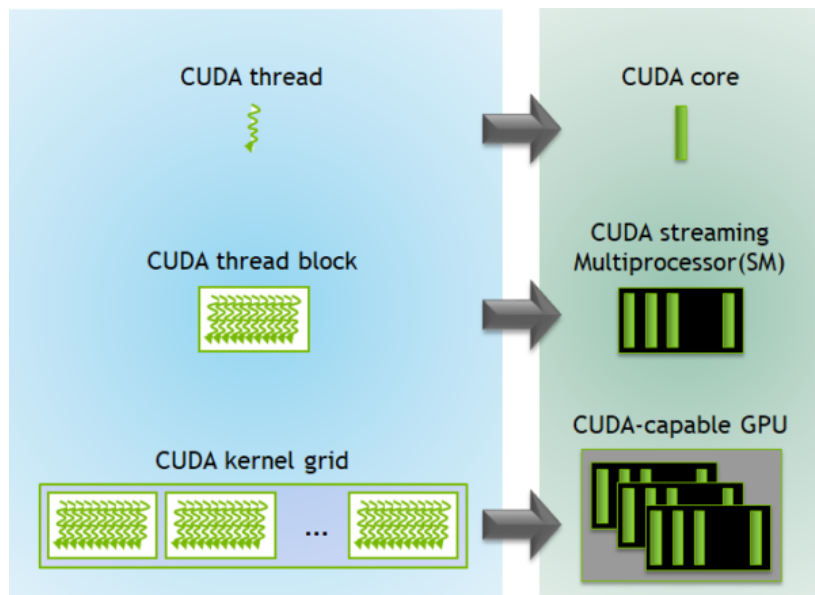


Figure 3.10 Cấu trúc phân cấp Grid, Block và Thread trong CUDA

3.4 Giao tiếp liên GPU: NCCL và NVSHMEM

Khi bài toán cần sức mạnh của nhiều GPU, giao tiếp giữa chúng quyết định hiệu năng hệ thống.

- **NCCL (NVIDIA Collective Communications Library):** Thư viện tối ưu hóa các phép toán giao tiếp tập thể (như All-Reduce, All-Gather) dành riêng cho nền tảng phần cứng NVIDIA (PCIe, NVLink, InfiniBand). NCCL được tích hợp sâu vào PyTorch DDP để đồng bộ gradient.

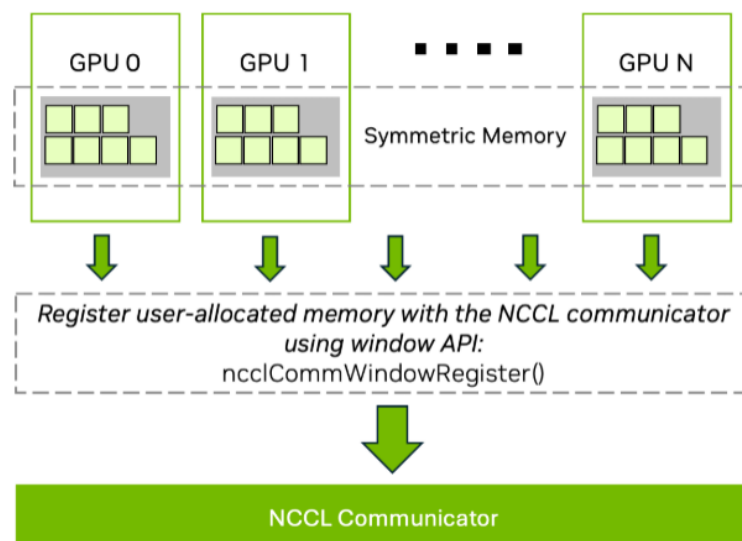


Figure 3.11 Giao tiếp tập thể sử dụng thư viện NCCL trên nhiều GPU

- **NVSHMEM:** Dựa trên tiêu chuẩn OpenSHMEM, NVSHMEM sử dụng mô hình

lập trình PGAS (Partitioned Global Address Space) và bộ nhớ vùng nhớ đối xứng (symmetric heap). Với giao tiếp truyền-nhận một phía (Put/Get), một GPU có thể ghi dữ liệu thẳng vào VRAM của GPU khác mà không cần sự can thiệp của CPU, đem lại độ trễ cực thấp dù đòi hỏi kiểm soát đồng bộ khắt khe (Barriers, Atomics).

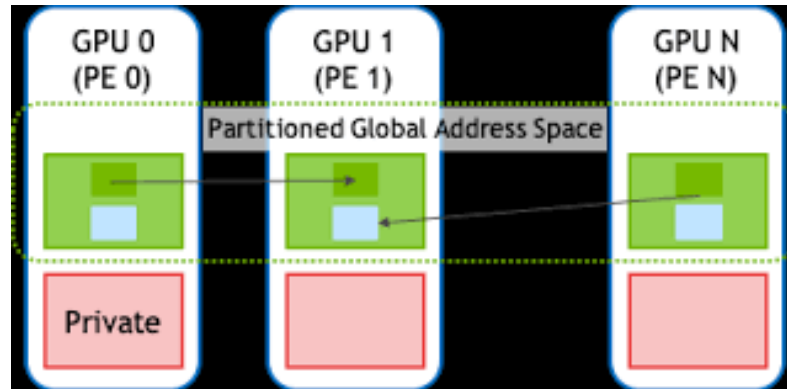


Figure 3.12 Mô hình bộ nhớ chia sẻ toàn cục phân vùng PGAS của NVSHMEM

4 Hệ thống huấn luyện và suy luận phân tán

4.1 Tổng quan về quá trình huấn luyện và suy luận mô hình AI

Quá trình phát triển và vận hành các mô hình ngôn ngữ lớn (LLM) bao gồm hai giai đoạn cốt lõi: huấn luyện (training) và suy luận (inference). Mặc dù cả hai giai đoạn đều chạy trên hệ thống phần cứng tính toán hiệu năng cao (GPU/VRAM) và sử dụng các chiến lược phân tán tương tự nhau, chúng có những đặc thù hệ thống và yêu cầu tối ưu hoàn toàn trái ngược:

- **Huấn luyện (Training):** Là quá trình học để tìm ra bộ trọng số tối ưu từ tập dữ liệu khổng lồ. Giai đoạn này đòi hỏi năng lực tính toán cực kỳ lớn cho cả chu trình truyền xuôi (forward pass) và truyền ngược (backward pass), đồng thời phải lưu trữ các trạng thái trung gian (gradients, optimizer states, activations). Sự đồng bộ liên tục qua mạng bằng thông rộng (như NVLink, InfiniBand) là bắt buộc.
- **Suy luận (Inference):** Là pha áp dụng mô hình đã huấn luyện hoàn chỉnh để tạo ra kết quả dựa trên các yêu cầu (requests) đầu vào thực tế từ người dùng. Quá trình tính toán nhẹ nhàng hơn và không có truyền ngược (no backpropagation), nhưng lại cực kỳ nhạy cảm với độ trễ (latency), thời gian phản hồi (Time-to-First-Token) và thông lượng (throughput) khi phục vụ đồng thời nhiều người dùng.

Bảng 4.3 so sánh chi tiết sự khác biệt về mặt hệ thống giữa quá trình Huấn luyện và Suy luận.

Table 4.3 So sánh đặc điểm hệ thống giữa Huấn luyện và Suy luận

Tiêu chí	Huấn luyện Training	Suy luận Inference
Yêu cầu tính toán	Cần năng lực tính toán khổng lồ gồm các thành phần như Gradient, Optimizer, chu trình Forward và Backward pass.	Tính toán nhẹ nhàng hơn, chủ yếu ở bước Prefill xây dựng KV Cache.
Áp lực bộ nhớ	Lưu trữ các biến trạng thái trung gian lớn như Activation Checkpointing.	Áp lực phụ thuộc chủ yếu vào độ dài chuỗi tạo ra KV Cache.
Giao tiếp & Mạng	Cần đồng bộ liên tục toàn bộ cụm, băng thông giao tiếp lớn.	Yêu cầu kết nối inter-node thấp hơn, chú trọng phân tải Load Balancing.
Sử dụng DP & FSDP	FSDP chia nhỏ bộ nhớ phổ biến, các GPU liên tục cần All-Reduce đồng bộ.	Bản sao DP xử lý các request độc lập, FSDP không được áp dụng.
Lượng tử hóa	Rất hạn chế do nguy cơ over/underflow mất độ chính xác trọng số.	Phổ biến như lượng tử hóa trọng số hoặc KV Cache để tăng tốc và giảm VRAM.
Mục tiêu hiệu năng	Tối ưu thời gian hoàn thành số Epoch.	Tối ưu độ trễ Latency, Time-to-First-Token, Thông lượng Throughput.

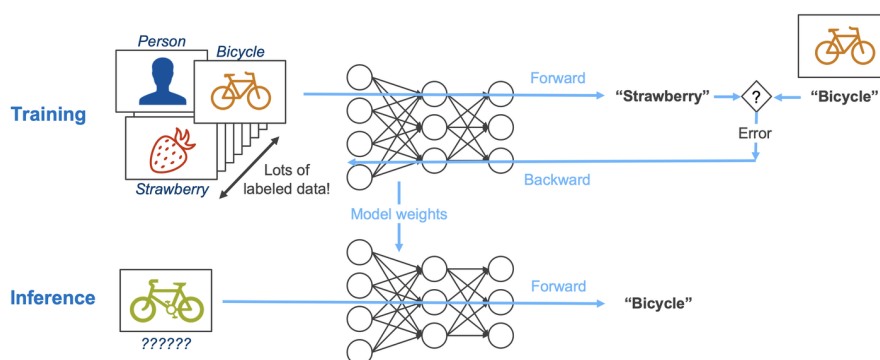


Figure 4.13 Sự khác biệt về đặc thù hệ thống và tài nguyên giữa quá trình huấn luyện và suy luận

Hệ sinh thái mã nguồn mở phục vụ cho cả hai quá trình trên được dẫn dắt bởi các framework huấn luyện phân tán mạnh mẽ và các engine suy luận chuyên biệt có hiệu năng cao.

4.1.1 Các Framework huấn luyện phân tán

Các framework huấn luyện phân tán chủ chốt bao gồm:

- **PyTorch Distributed:** Cung cấp sẵn các lớp cơ sở hạ tầng như DDP, FSDP, TP giúp dễ dàng khởi chạy, cắt nhỏ dữ liệu hoặc chia khối mô hình cho các GPU.
- **DeepSpeed từ Microsoft [1]:** Hoạt động như lớp trung gian tối ưu hóa, nổi tiếng với công nghệ ZeRO giúp giảm đáng kể áp lực bộ nhớ. Nó mở rộng từ huấn luyện sang tối ưu hóa suy luận như DeepSpeed-FastGen, DeepSpeed-Chat.
- **Megatron-LM / Megatron Core của NVIDIA:** Mã nguồn tiêu chuẩn công nghiệp cho việc huấn luyện quy mô lớn, tích hợp các kiến trúc song song tiên tiến và hiệu quả nhất cho các chip của NVIDIA.
- **Ray Train [2]:** Quản lý tài nguyên, điều phối quá trình huấn luyện ở quy mô cụm cluster.

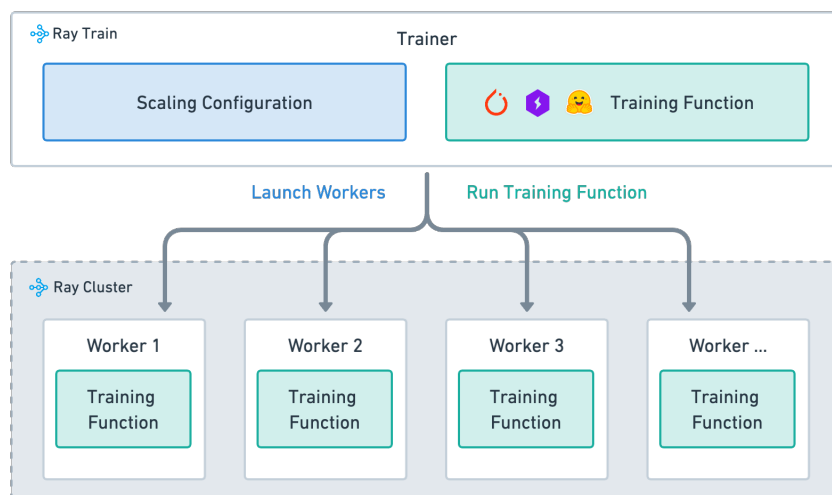


Figure 4.14 Kiến trúc quản lý và điều phối tài nguyên của Ray Train trong huấn luyện phân tán

4.1.2 Các Engine suy luận phân tán

Bên cạnh quá trình huấn luyện, vLLM và SGLang là hai engine mã nguồn mở phục vụ suy luận (inference) LLM hàng đầu hiện nay, đóng vai trò quan trọng trong việc tối ưu hóa tốc độ sinh token và hiệu suất sử dụng bộ nhớ:

- **vLLM [3]:** Tiên phong đề xuất cơ chế PagedAttention để giải quyết vấn đề phân mảnh bộ nhớ của KV Cache. Engine này cho phép chia nhỏ và quản lý động KV Cache tương tự như cách hệ điều hành quản lý bộ nhớ ảo, giúp tăng đáng kể thông lượng phục vụ đồng thời nhiều yêu cầu sinh văn bản.

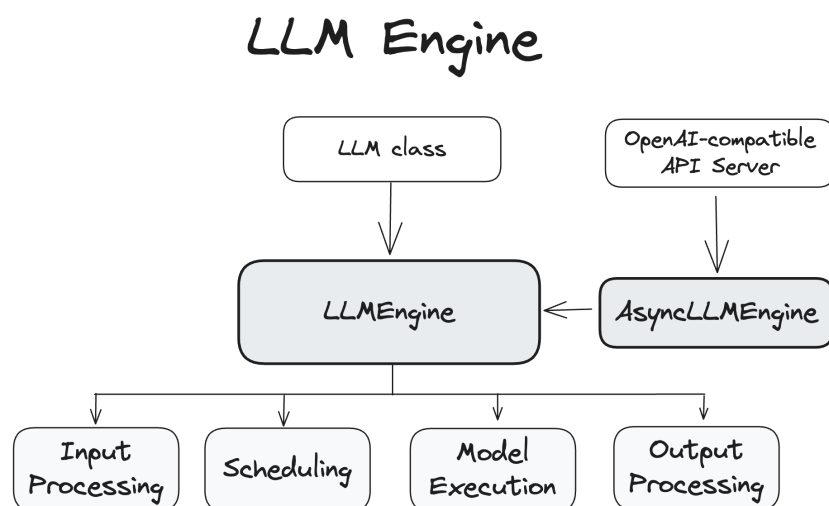


Figure 4.15 Sơ đồ phân bổ và quản lý KV Cache của vLLM sử dụng cơ chế Page-dAttention

- **SGLang [4]:** Công cụ tối ưu hóa hiệu năng suy luận thông qua việc giới thiệu ngôn ngữ lập trình có cấu trúc Radix Attention. SGLang tối ưu hóa việc quản lý bộ nhớ KV Cache trên Radix Tree, cho phép chia sẻ KV Cache giữa nhiều tiến trình và yêu cầu có phần tiền tố chung, giúp giảm thời gian sinh token ban đầu và tăng tốc độ xử lý các prompt phức tạp.

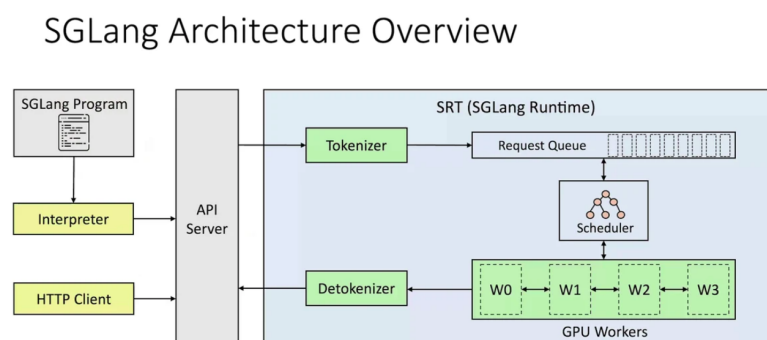


Figure 4.16 Kiến trúc của SGLang

4.2 Các chiến lược song song hóa

Việc phân bổ mô hình qua nhiều GPU phụ thuộc vào nhiều chiều không gian cắt Sharding. Bảng 4.4 tóm tắt các kỹ thuật nổi bật nhất.

Table 4.4 Tổng hợp các chiến lược song song hóa trên mô hình AI

Chiến lược	Đặc điểm hoạt động & Cơ chế
DP Data	Mỗi GPU giữ bản sao nguyên vẹn mô hình, xử lý data batch riêng rẽ. Gộp Gradient bằng Bucketed Ring All-Reduce.
FSDP / ZeRO	Băm nhỏ trọng số Weights, Gradient, Optimizer chia qua các GPU. All-Gather để kéo trọng số về tính toán, sau đó Reduce-Scatter gộp kết quả và xóa vùng nhớ cục bộ.
PP Pipeline	Cắt ngang các tầng layer sâu của mô hình. Tối ưu bằng lịch trình 1F1B - One Forward, One Backward hoặc Interleaved để giảm Pipeline Bubble.
TP Tensor	Cắt ma trận tính toán theo chiều dọc để các GPU tính trên cùng một layer. Đòi hỏi tốc độ mạng cực cao để đồng bộ tức thì.
CP Context	Băm nhỏ ngữ cảnh chuỗi đầu vào. Giải quyết bài toán Attention xuyên GPU qua Ring Attention hoặc DeepSpeed Ulysses.
EP Expert / MoE	Đẩy dữ liệu input qua một mạng Gating Network để chia Token tới các GPU chuyên gia thích hợp qua giao tiếp All-to-All.

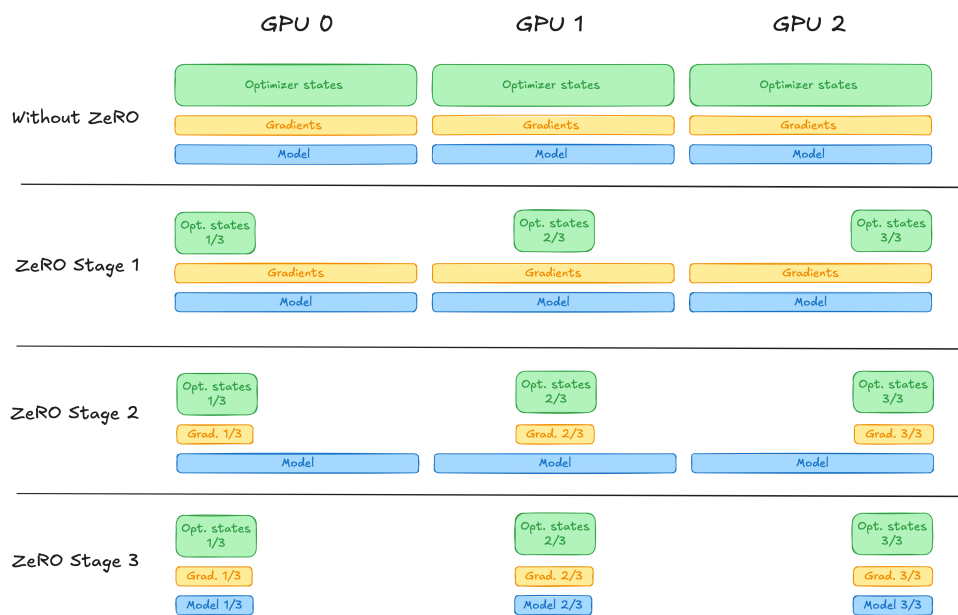


Figure 4.17 Các cấp độ tối ưu hóa bộ nhớ của DeepSpeed ZeRO

4.3 Tối ưu hóa quá trình suy luận Inference Optimization

Quá trình suy luận Inference của các mô hình ngôn ngữ lớn thường gặp hiện tượng nghẽn do giới hạn băng thông bộ nhớ memory-bound, đặc biệt ở pha sinh token liên tiếp Decode. Nhằm tối ưu hóa hiệu năng hệ thống, giảm thiểu độ trễ Latency và nâng cao thông lượng Throughput, các kỹ thuật tối ưu hóa được phát triển tập trung vào ba pha và khía cạnh cốt lõi: pha Prefill xử lý prompt đầu vào, quản lý bộ nhớ đệm KV Cache, và

pha Decode tự hồi quy sinh token tiếp theo.

4.3.1 Tối ưu hóa trong pha Prefill

Pha Prefill chịu trách nhiệm tính toán các biểu diễn ban đầu cho toàn bộ chuỗi prompt đầu vào, có tính chất tính toán nặng compute-bound. Các kỹ thuật tối ưu nổi bật bao gồm:

- **FlashAttention:** Đây là kỹ thuật tối ưu hóa tính toán Attention ở mức phần cứng. Thay vì tính toán và lưu trữ ma trận Attention đầy đủ kích thước $N \times N$ với N là chiều dài chuỗi lên bộ nhớ băng thông cao HBM gây nghẽn băng thông, FlashAttention chia nhỏ ma trận thành các khối tiling để tính toán trực tiếp trong bộ nhớ đệm tốc độ cực cao SRAM của GPU. Bằng việc áp dụng kỹ thuật *online softmax*, thuật toán thực hiện chuẩn hóa softmax mà không cần lưu trữ các kết quả trung gian khổng lồ ra HBM, giảm đáng kể tần suất đọc/ghi và tiết kiệm tài nguyên bộ nhớ đệm.

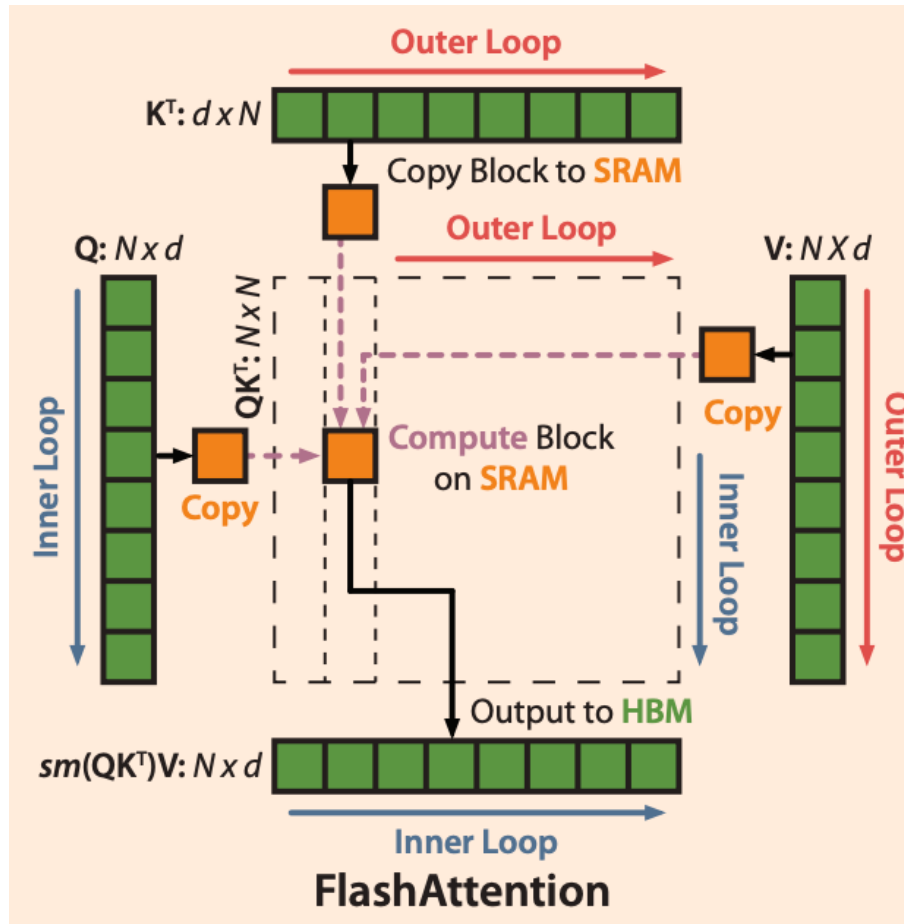


Figure 4.18 Cơ chế chia khối Tiling và tính toán trực tiếp trên SRAM của FlashAttention

- **Chunked Prefill:** Đối với các prompt đầu vào có độ dài cực lớn, pha Prefill có thể chiếm dụng toàn bộ tài nguyên tính toán của GPU trong thời gian dài, gây ra hiện

tượng nghẽn cổ chai blocking. Kỹ thuật Chunked Prefill giải quyết vấn đề này bằng cách chia prompt dài thành các phân đoạn chunk nhỏ hơn. Các chunk này được lên lịch xử lý xen kẽ với các batch decode của các yêu cầu requests khác đang chạy, ngăn ngừa tình trạng các yêu cầu decode bị "đói" tài nguyên bộ nhớ hoặc phải chờ đợi quá lâu.

- **Prefix Caching:** Tận dụng việc nhiều requests có chung phần tiền tố (như system prompt hoặc ngữ cảnh RAG) để lưu trữ sẵn và tái sử dụng KV Cache. Cơ chế hoạt động dựa trên cấu trúc cây *Radix Tree*. Khi prompt mới đi vào, hệ thống đối chiếu mảng Token ID với cây Radix; nếu trùng khớp tiền tố, hệ thống lấy ngay KV Cache của nhánh đó mà không cần GPU tính toán lại, giúp GPU chỉ cần xử lý pha Prefill cho phần nội dung mới phía sau.

4.3.2 Tối ưu hóa quản lý bộ nhớ đệm KV Cache

KV Cache lưu trữ các vector Key và Value của các token đã xử lý để tránh tính toán lặp lại trong pha Decode. Tuy nhiên, kích thước KV Cache tăng tuyến tính theo chiều dài chuỗi sinh ra và số lượng yêu cầu đồng thời, dẫn đến nguy cơ cạn kiệt bộ nhớ VRAM. Các phương pháp quản lý bộ nhớ đệm hiệu quả gồm có:

- **PagedAttention:** Tương tự như cơ chế quản lý trang nhớ trong hệ điều hành, PagedAttention chia nhỏ KV Cache của mỗi yêu cầu thành các trang vật lý *Pages* có kích thước cố định và có thể nằm rải rác trên VRAM. Một bảng quản lý khối *Block Table* đóng vai trò ánh xạ giữa vị trí logic của token và địa chỉ vật lý của trang tương ứng. Khi cần truy xuất dữ liệu, hệ thống thông qua Block Table để trở tới trang thích hợp. Khi sinh ra token mới, hệ thống chỉ việc cấp phát một trang trống bất kỳ mà không cần tìm kiếm các phân vùng nhớ liên tục lớn, triệt tiêu phân mảnh bộ nhớ và tối ưu hiệu suất sử dụng VRAM.
- **RadixAttention:** Dùng cấu trúc cây Radix để theo dõi luồng request đi vào, tự động phát hiện các đoạn văn bản (prompt) trùng lặp giữa các request khác nhau. Từ đó, nó đưa ra quyết định tái sử dụng: yêu cầu GPU không tính toán lại đoạn trùng lặp mà dùng chung KV Cache đã có.
- **Multi-head Latent Attention MLA:** Thay vì tính toán và lưu trữ các ma trận Key và Value độc lập cho từng Head đầu Attention vào VRAM, MLA nén toàn bộ thông tin này xuống một vector nén siêu nhỏ *Latent vector* thông qua một ma trận chiếu thấp chiều Down-projection và chỉ lưu vector này vào VRAM. Trong pha Decode, vector nén sẽ đi qua lớp giải nén tuyến tính Up-projection trực tiếp trên bộ nhớ SRAM của GPU để tái tạo lại các ma trận Key và Value trước khi tính toán. Điều

này giúp giảm đáng kể footprint bộ nhớ của KV Cache trên VRAM mà không làm suy giảm chất lượng mô hình.

- **KV Compression và Quantization:** Thực hiện nén và lượng tử hóa KV Cache để tiết kiệm bộ nhớ. Bằng việc phân tích điểm số Attention *Attention Score* ở các tầng trước đó, hệ thống có thể gộp hoặc loại bỏ các token có tầm quan trọng thấp ít được chú ý. Đồng thời, việc áp dụng lượng tử hóa độ chính xác thấp như FP8, INT4 hoặc phân tách low-rank cho KV Cache giúp giảm dung lượng lưu trữ trên mỗi token.

4.3.3 Tối ưu hóa trong pha Decode

Pha Decode chịu trách nhiệm sinh tự hồi quy từng token một, có đặc điểm là tính toán nhẹ nhưng đòi hỏi truy xuất bộ nhớ liên tục memory-bound. Các phương pháp tối ưu tập trung tăng thông lượng và giảm số lần gọi mô hình:

- **Continuous Batching [5]:** Thay vì sử dụng batching tĩnh, phải đợi tất cả các request trong batch hoàn thành mới bắt đầu batch mới, Continuous Batching hay *Iteration-level Scheduling* gom các yêu cầu đang ở trạng thái decode vào một batch động. Sau mỗi bước sinh token mỗi iteration, bộ điều phối scheduler sẽ kiểm tra trạng thái: loại bỏ các yêu cầu đã sinh xong nhận token kết thúc chuỗi và thêm các yêu cầu mới đã hoàn thành pha Prefill vào batch. Điều này giúp GPU luôn hoạt động với hiệu suất tối đa và đáp ứng dòng yêu cầu đến liên tục.
- **Speculative Decoding (Giải mã suy đoán) [6]:** Sử dụng một mô hình nhỏ, chạy nhanh *Draft Model* để sinh trước một chuỗi gồm nhiều token dự kiến. Sau đó, mô hình lớn chính *Target Model* sẽ kiểm tra toàn bộ chuỗi token này cùng lúc trong một lần chạy Forward duy nhất. Các token hợp lệ phù hợp với phân phối xác suất của mô hình lớn sẽ được chấp nhận, các token sai lệch sẽ bị loại bỏ và sinh lại. Phương pháp này giảm thiểu đáng kể số lần kích hoạt mô hình lớn, tuy nhiên có nhược điểm là phải duy trì đồng thời hai mô hình trong bộ nhớ, và nếu sự khác biệt phân phối distribution mismatch giữa hai mô hình lớn, tỷ lệ từ chối *reject rate* sẽ cao, làm giảm hiệu quả tối ưu.
- **Medusa [7]:** Giải quyết điểm yếu của Speculative Decoding truyền thống bằng cách loại bỏ Draft Model riêng biệt. Medusa tích hợp thêm nhiều đầu dự đoán phụ *Medusa Heads* vào trực tiếp mô hình lớn để dự đoán đồng thời nhiều token tiếp theo từ cùng một trạng thái ẩn hidden state. Chuỗi các token dự đoán này được cấu trúc dưới dạng cây *Tree Attention* và đưa vào mô hình chính để xác thực song song, chọn ra nhánh có xác suất tốt nhất.

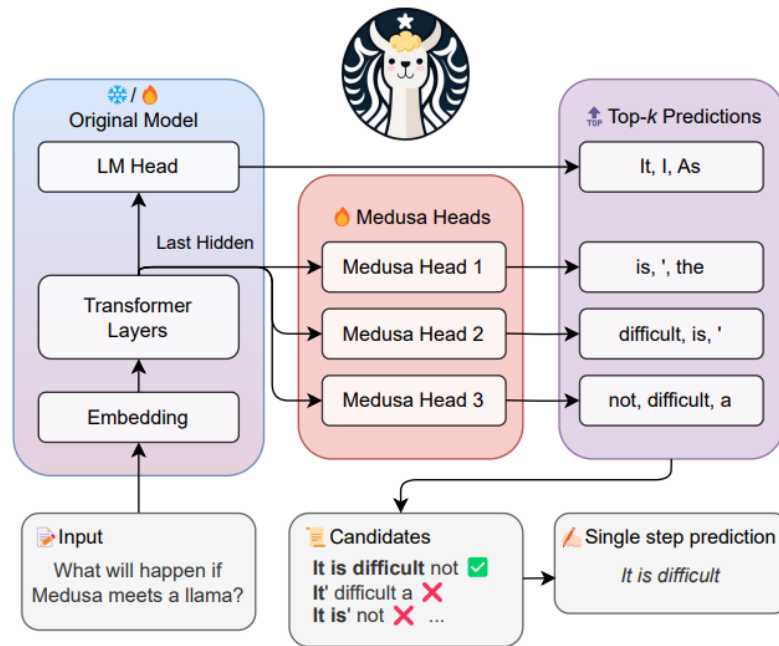


Figure 4.19 Kiến trúc sinh token song song của Medusa sử dụng Tree Attention

- **EAGLE Feature-level Speculation [8]:** Là một cải tiến sâu hơn của giải mã suy đoán. Thay vì dự đoán ở mức token dễ sai lệch và mất thông tin, EAGLE thực hiện suy đoán ở cấp độ đặc trưng *Feature-level/Hidden-state*. Kỹ thuật này sử dụng một kiến trúc tự hồi quy siêu nhẹ chạy trên chuỗi vector đặc trưng của mô hình lớn để sinh bản thảo draft features, mang lại độ chính xác cao hơn và tỷ lệ chấp nhận vượt trội trong thực tế.

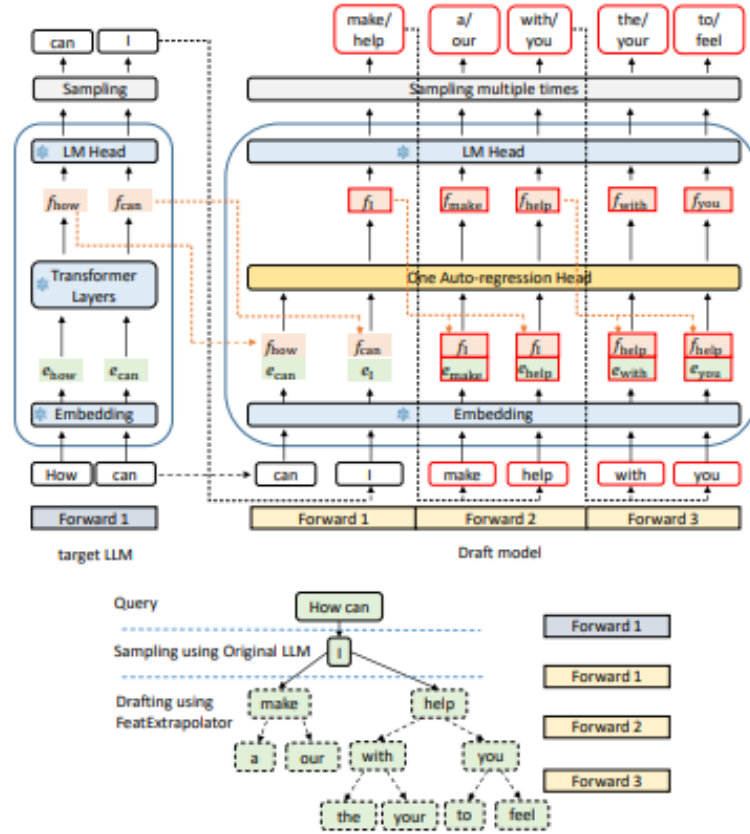


Figure 4.20 Cơ chế Speculative Decoding ở cấp độ Feature của EAGLE

- **Multi-Token Prediction (MTP) [9]:** Kỹ thuật này thay đổi cách huấn luyện mô hình từ pha tiền huấn luyện Pre-training. Mô hình được thiết kế để dự đoán đồng thời nhiều token tương lai tại mỗi vị trí, thay vì chỉ token tiếp theo. Bên cạnh việc nâng cao chất lượng biểu diễn nội tại của mô hình, các đầu dự đoán tương lai này có thể được sử dụng trực tiếp để thực hiện Speculative Decoding mà không cần bất kỳ mô hình phụ hay đầu cây ghép bổ sung nào.

4.4 Kết luận

Chương này đã đi sâu vào cách kết hợp và hiện thực hóa các lý thuyết song song thông qua các Framework tiên tiến nhất. Nghiên cứu cũng chỉ ra rằng việc tối ưu hệ thống cần xem xét đến các khác biệt cốt lõi giữa hai pha huấn luyện và suy luận để áp dụng các chiến lược như DP, PP, FSDP hay Quantization một cách hiệu quả nhất.

5 Thí nghiệm minh họa (Dự kiến)

Phần thí nghiệm dự kiến nhằm đánh giá thực tế tác động của các kỹ thuật giúp cải thiện quá trình Inference, so sánh hiệu năng Inference qua các chỉ số như time to first token, token/s ..., chạy Inference một mô hình lớn không thể nằm gọn trong 1 GPU. Phân tích khả năng serve request của hệ thống.

Môi trường dự kiến: Kaggle, 2 GPU T4.

TÀI LIỆU THAM KHẢO

- [1] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, “Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 3505–3506.
- [2] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, W. Paul, M. I. Jordan, and I. Stoica, “Ray: A distributed framework for emerging AI applications,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 561–577.
- [3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.
- [4] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “Sglang: Efficient execution of structured language model programs,” *arXiv preprint arXiv:2312.07104*, 2023.
- [5] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [6] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 19 274–19 286.
- [7] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple LLM inference acceleration framework with multiple decoding heads,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [8] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle: Speculative sampling requires rethinking feature uncertainty,” in *International Conference on Machine Learning (ICML)*, 2024.
- [9] F. Gloeckle, A. Bapna, V. Balachandran, L. Miculicich, A. Bapna *et al.*, “Better & faster large language models via multi-token prediction,” *arXiv preprint arXiv:2404.19737*, 2024.

- [10] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania *et al.*, “Pytorch distributed: Experiences on accelerating data parallel training,” *arXiv preprint arXiv:2006.15704*, 2020.
- [11] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “Zero: Memory optimizations toward training trillion parameter models,” *arXiv preprint arXiv:1910.02054*, 2019.
- [12] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” *arXiv preprint arXiv:1909.08053*, 2019.
- [13] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *arXiv preprint arXiv:2205.14135*, 2022.
- [14] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ng, and Q. V. Le, “Gpipe: Efficient training of giant neural networks using pipeline parallelism,” *arXiv preprint arXiv:1811.06965*, 2018.
- [15] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, G. A. Gibbons, and M. Zaharia, “Pipedream: Generalized pipeline parallelism for dnn training,” *arXiv preprint arXiv:1906.02243*, 2019.
- [16] D. B. Kirk and W.-m. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2016.
- [17] S. University, “Cme 213: Introduction to parallel computing using mpi, openmp, and cuda,” <https://github.com/cme213/cme213-spring-2021>, 2021.
- [18] L. L. N. Laboratory, “Llnl hpc documentation,” <https://hpc.llnl.gov/documentation>.
- [19] NVIDIA, “Nvidia documentation center,” <https://docs.nvidia.com/>.
- [20] “Gpu mode,” <https://www.gpumode.com/>.

PHỤ LỤC

Phụ lục A: Cấu hình môi trường thực nghiệm dự kiến

(Nội dung mã nguồn khởi tạo môi trường, script cài đặt các thư viện như PyTorch Distributed, DeepSpeed và Megatron-Core sẽ được cập nhật sau khi tiến hành thực nghiệm cụ thể).

Phụ lục B: Log chạy benchmark PyTorch DDP

(Kết quả log hiển thị từ terminal khi so sánh thời gian chạy 1 epoch trên 1 GPU so với 2 GPU sẽ được đính kèm tại đây để làm minh chứng cho kết quả trong Chương 4).